

Proyecto de chat entre Cliente-Servidor

Jonathan

24 de septiembre de 2026

Resumen

En este proyecto se realizará un chat entre varios clientes mediante un servidor, donde los clientes podrán comunicarse de forma ordinaria utilizando varios comandos de fácil intuición. Al iniciar, los usuarios deberán identificarse, y una vez dentro podrán enviarse mensajes entre ellos, ya sea públicos o privados. Además, podrán pedir la lista de usuarios conectados, cambiar su estado (activo, ocupado, etc.), y crear salas privadas a las que podrán invitar a otros usuarios y a las cuales estos podrán unirse.

Índice

1. Introducción	2
2. Análisis y diseño	2
2.1. Planteamiento del problema	2
2.2. Inicio de sesión e identificación	2
2.3. Comunicación pública y privada	2
2.4. Estados de los usuarios	3
2.5. Consulta de usuarios	3
2.6. Creación y utilización de salas privadas	3
2.7. Abandono de salas y desconexión de usuarios	3
2.8. Diseño	4
3. Implementación	6
3.1. Tecnologías utilizadas	6
3.2. El protocolo de comunicación	6
3.3. Concurrencia: un hilo por cliente	6
3.4. Arquitectura usada	7
3.5. Validación y manejo de errores	7
3.6. Pruebas	7
3.7. Experiencia de usuario en el cliente	8
4. Conclusiones	8
5. Bibliografía	8

1. Introducción

Este proyecto está hecho, más que nada, para ponernos a nosotros mismos un reto: aprender un lenguaje nuevo, salir de la rutina y aprender conforme lo vayamos necesitando. También busca poner en práctica algunos temas del curso, como el análisis y diseño (que se abordará más adelante), la implementación, la distribución e instalación, y buenas prácticas de programación como las pruebas unitarias, el código limpio y los patrones de diseño, que nos sirvieron en el proyecto para no tener toda la lógica amontonada en el `main`.

Este proyecto se parece, en pequeño, a cómo funcionan a gran escala aplicaciones como WhatsApp, Messenger, Discord o Teams, y con él lo estamos viendo desde más adentro.

Elegí Kotlin porque ya me sentía demasiado familiarizado con Java, y Kotlin es un lenguaje muy similar a este, además de que es orientado a objetos y es fácil de aprender.

2. Análisis y diseño

2.1. Planteamiento del problema

El problema principal que se busca resolver es la creación de un sistema que permita establecer una comunicación entre varios usuarios de manera organizada. Para lograrlo, es necesario considerar diferentes reglas y funcionalidades que permitan controlar las acciones que puede realizar cada usuario. El sistema debe ser capaz de identificar a cada persona conectada, conocer su estado, permitirle enviar mensajes y controlar su acceso a las diferentes salas.

2.2. Inicio de sesión e identificación

Al conectarse por primera vez, el usuario debe proporcionar un nombre o apodo mediante el cual será identificado dentro del chat, el cual no debe ser de más de 8 caracteres. Este nombre debe ser único entre los usuarios conectados, de manera que no puedan existir dos personas utilizando exactamente el mismo nombre al mismo tiempo. Si se intenta ingresar uno que ya está en uso, no se permite la entrada al chat y se debe solicitar otro nombre.

Una vez que el usuario se identifica correctamente, los demás usuarios conectados reciben una notificación indicando que un nuevo usuario se ha unido al chat. El usuario comienza su participación con un estado activo, y a partir de ese momento puede realizar las diferentes acciones permitidas por el sistema.

2.3. Comunicación pública y privada

Al encontrarse en el chat general, el usuario puede escribir un mensaje que, por defecto, está dirigido a todos los usuarios conectados, sin necesidad de seleccionar un destinatario.

Además de los mensajes públicos, el sistema permite establecer conversaciones privadas entre dos usuarios. En caso de que el usuario busque a una persona que no se encuentre conectada, el sistema informa que no existe actualmente un usuario disponible con ese nombre, evitando que el mensaje se intente enviar a un destinatario que no puede recibirlo.

2.4. Estados de los usuarios

Otra característica importante es la posibilidad de establecer un estado que indique la disponibilidad de cada usuario, permitiendo que los demás participantes conozcan si una persona se encuentra disponible, ausente u ocupada. Se contemplan tres estados: Activo (**ACTIVE**), Ausente (**AWAY**) y Ocupado (**BUSY**).

Cuando un usuario modifica su estado, los demás participantes conectados reciben una notificación del cambio, de forma que la información se mantiene actualizada.

2.5. Consulta de usuarios

Cualquier usuario conectado puede solicitar la lista de usuarios, ya sea la del chat general o la de una sala privada a la que pertenezca. En ambos casos, el sistema muestra el nombre de cada usuario junto con su estado actual.

2.6. Creación y utilización de salas privadas

Además de la conversación general, el sistema permite crear salas privadas destinadas a grupos específicos de usuarios. Para crear una sala, el usuario debe proporcionar un nombre; el protocolo establece un máximo de 16 caracteres, y si el nombre ya está en uso por otra sala activa, el sistema lo informa y no permite la creación.

El usuario que crea una sala es inicialmente su único integrante. Posteriormente puede invitar a otros usuarios, quienes reciben una notificación de invitación y pueden incorporarse a la sala. Una vez dentro, cualquier integrante puede a su vez invitar a más usuarios, siguiendo las reglas establecidas.

Una característica importante es que los usuarios no pueden expulsar a otros integrantes de una sala; si alguien fue invitado por error, deberá abandonar la sala por su cuenta. También es posible consultar los integrantes de una sala en cualquier momento.

Finalmente, cuando todos los usuarios abandonan una sala, esta deja de existir, y será necesario crear nuevamente una sala con ese nombre si se desea utilizarlo posteriormente.

2.7. Abandono de salas y desconexión de usuarios

Cuando un usuario decide abandonar una sala privada, se muestra un aviso dentro de la misma indicando que dicho usuario la abandonó; este deja de aparecer en la lista de integrantes de esa sala, aunque continúa conectado al chat general.

La desconexión completa del chat es distinta: en este caso, el usuario deja de aparecer en la lista general de usuarios conectados y también abandona las salas privadas en las que se encontraba. Los demás usuarios reciben un aviso indicando que la persona se desconectó.

También se contemplan las situaciones en las que un usuario intenta realizar una acción no válida, como enviar un mensaje privado a alguien que no está conectado, o una acción relacionada con una sala que ya no existe; el sistema informa al usuario sobre lo ocurrido. En los casos en los que se recibe una instrucción inválida o que incumple las reglas del protocolo, el usuario es desconectado del sistema, y se muestra el aviso correspondiente al resto de los usuarios.

2.8. Diseño

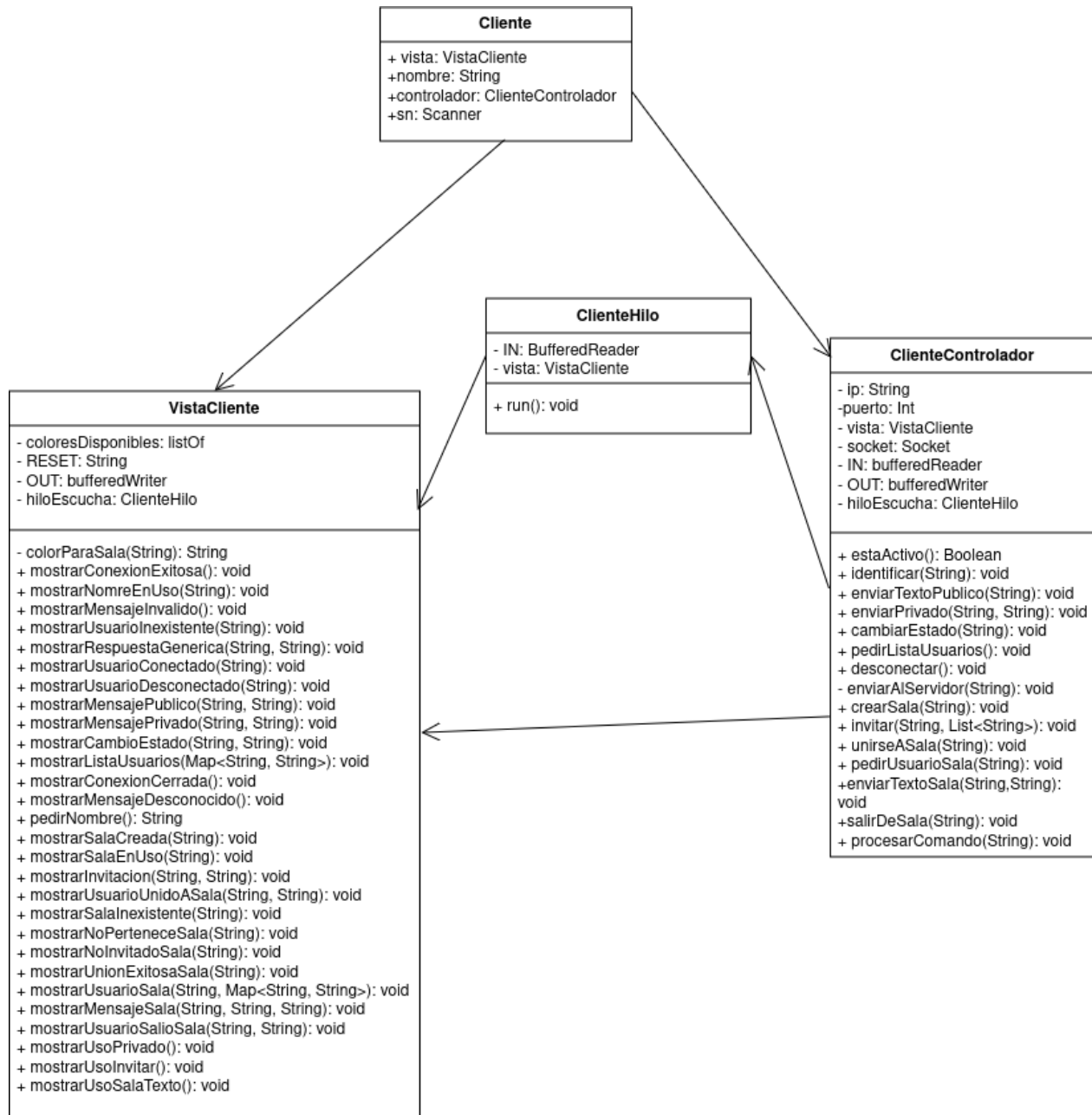


Figura 1: Diagrama de clases del cliente.

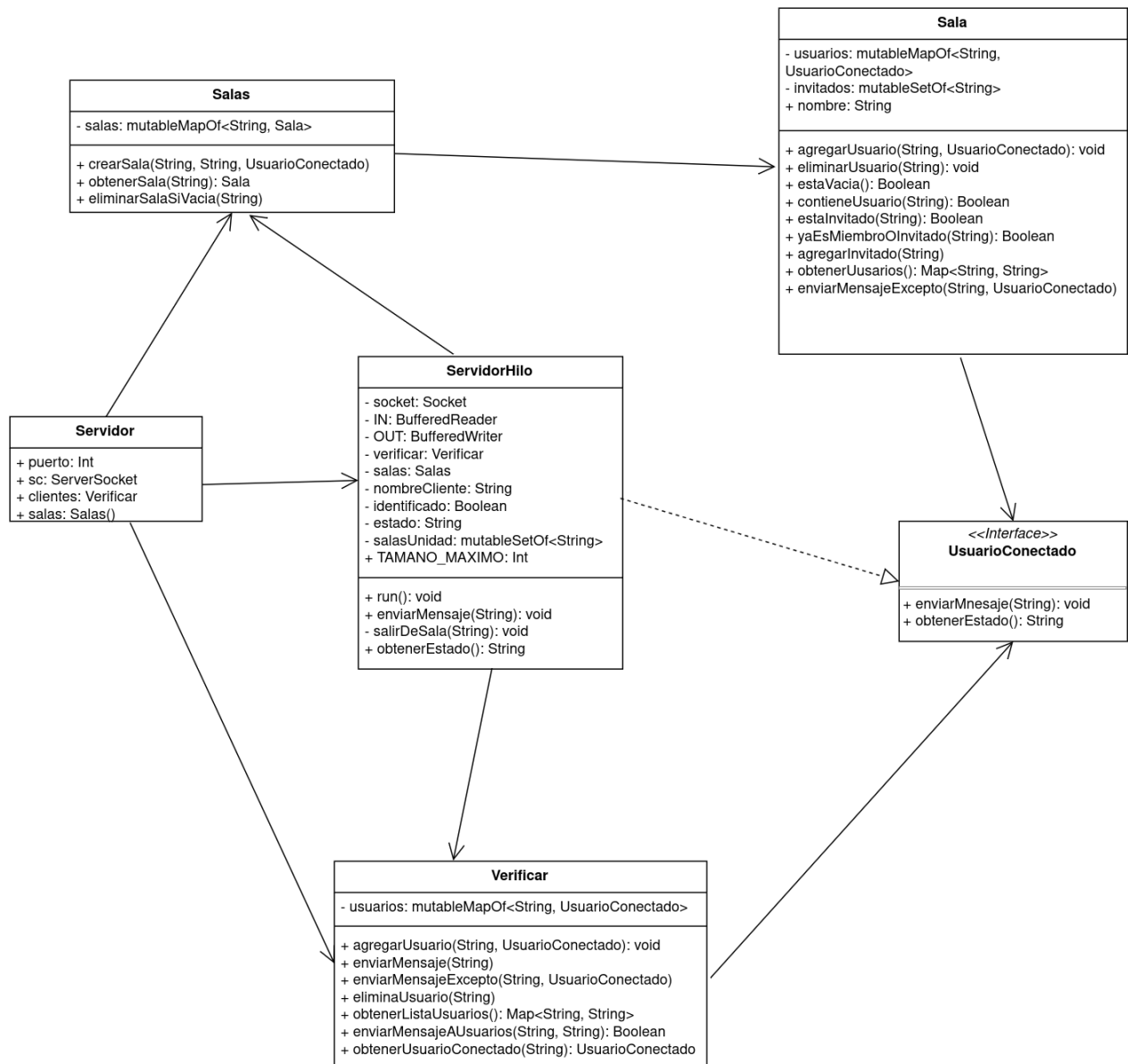


Figura 2: Diagrama de clases del servidor.

3. Implementación

3.1. Tecnologías utilizadas

Para este proyecto usé Kotlin, y para compilar y manejar las dependencias usé Maven. Aparte usé algunas bibliotecas que me ayudaron bastante: `kotlinx.serialization`, que se encarga de convertir mis clases de Kotlin a JSON y viceversa, sin tener que armar el texto a mano; los Sockets de Java (`Socket` y `ServerSocket`), para que cliente y servidor pudieran conectarse y mandarse información por TCP; y `Threads` de Java, para poder atender a varios clientes al mismo tiempo. También hice pruebas (tests) para verificar que el protocolo funcionara bien, conectándome de verdad por sockets a un servidor real.

3.2. El protocolo de comunicación

Cliente y servidor se hablan mandándose mensajes de texto en formato JSON, y cada mensaje termina con un salto de línea, que sirve para saber dónde acaba uno y empieza el siguiente. Todo mensaje trae un campo `type` que dice de qué se trata (por ejemplo `IDENTIFY`, `PUBLIC_TEXT`, `NEW_ROOM`), y con eso el servidor decide qué hacer. Por ejemplo, así se ve cuando alguien se identifica y manda un mensaje público:

```
// Cliente -> Servidor
{"type":"IDENTIFY","username":"Jonathan"}

// Servidor -> Cliente
{"type":"RESPONSE","operation":"IDENTIFY","result":"SUCCESS","extra":"Jonathan"}

// Servidor -> Resto de los clientes
{"type":"NEW_USER","username":"Jonathan"}
```

Cada tipo de mensaje lo modelé como una `data class` de Kotlin marcada con `@Serializable`, así la biblioteca se encarga sola de convertirla a JSON y de regreso, y yo no tengo que armar el texto a mano ni arriesgarme a equivocarme con una coma o una comilla.

```
@Serializable
data class PublicTextFrom(
    val type: String = "PUBLIC_TEXT_FROM",
    val username: String,
    val text: String
)
```

3.3. Concurrencia: un hilo por cliente

El servidor está esperando conexiones en un ciclo infinito. Cada vez que alguien se conecta, se crea un hilo nuevo (`ServidorHilo`) solo para atender a ese cliente, y se deja corriendo en paralelo. Así el servidor puede seguir aceptando más gente sin tener que esperar a que termine de hablar con el cliente anterior.

Todos esos hilos comparten un mismo objeto `Verificar` (con la lista de usuarios conectados) y un mismo objeto `Salas` (con las salas activas). Como varios hilos pueden estar leyendo y modificando esa información al mismo tiempo, tuve que marcar todos sus métodos como `@Synchronized`, para que solo un hilo a la vez pueda tocar esos datos. De hecho, esto lo descubrí gracias a una prueba de estrés que hice con 100 usuarios conectándose a la vez: sin esa protección, el programa tronaba con errores de concurrencia (`ConcurrentModificationException`) porque un hilo modificaba la lista mientras otro la estaba recorriendo.

3.4. Arquitectura usada

Organicé tanto el cliente como el servidor con el patrón Modelo-Vista-Controlador. El Modelo son las clases del protocolo, más `Verificar` y `Salas`, que guardan el estado real del sistema (quién está conectado, qué salas existen). La Vista, solo del lado del cliente, es `VistaCliente`, la única parte del programa que muestra texto en pantalla. Y el Controlador son `ClienteControlador` y `ClienteHilo` del lado del cliente, y `ServidorHilo` del lado del servidor, que coordinan entre la Vista y el Modelo.

Un detalle que tuve que resolver: `Verificar` necesita poder mandarle mensajes a cada usuario conectado, pero la clase que representa a un usuario conectado (`ServidorHilo`) es parte del Controlador, no del Modelo. Para no romper la regla de que el Modelo nunca debe depender del Controlador, hice una interfaz `UsuarioConectado` (que sí es parte del Modelo), y `ServidorHilo` la implementa. Así, `Verificar` solo conoce la interfaz, nunca la clase concreta.

3.5. Validación y manejo de errores

El servidor revisa que cada mensaje que le llega sea JSON válido y que no se pase de 1 megabyte; si algo de esto falla, responde con un mensaje de tipo `INVALID` y desconecta a ese cliente. También valida los límites que pide el protocolo: nombres de usuario de máximo 8 caracteres y nombres de sala de máximo 16.

Para los errores que sí pueden pasar en el uso normal del chat como mandarle un mensaje privado a alguien que ya no está conectado, o intentar unirse a una sala que no existe el servidor responde con un error específico (por ejemplo `NO_SUCH_USER` o `NOT_INVITED`), pero sin desconectar a nadie, porque no es un error grave, solo una acción que no se pudo completar.

3.6. Pruebas

Para asegurarme de que todo funcionara bien, hice varias pruebas que se conectan de verdad al servidor por sockets, igual que lo haría un cliente real, y revisan que las respuestas sean las que el protocolo espera. Entre las que hice están:

- Conectarse con éxito, y que rechace un nombre repetido.
- Mandar y recibir mensajes privados, incluyendo cuando el destinatario no existe.
- Pedir la lista de usuarios conectados y ver sus estados.

- Mandar un mensaje que se pasa del tamaño máximo, y que el servidor lo rechace.
- El flujo completo de una sala: crearla, invitar, unirse, ver quién está, mandar texto, y salir.
- Una prueba de estrés conectando 100 usuarios al mismo tiempo, que fue la que me ayudó a encontrar el bug de concurrencia que mencioné antes.

3.7. Experiencia de usuario en el cliente

Para que fuera más fácil de usar desde la terminal, hice que el cliente reconociera comandos escritos entre signos de menor y mayor que, por ejemplo `<privado>usuario mensaje` o `<crear_sala>nombre`; cualquier otro texto que no sea un comando se manda como mensaje público. Si escribes mal un comando por ejemplo, `<privado>` sin poner el usuario y el mensaje el propio cliente se da cuenta antes de mandar nada al servidor, y te muestra cómo se debe escribir (por ejemplo “Uso: `<privado> usuario mensaje`”), para que no se te vaya un mensaje mal formado por accidente. También, para que fuera más fácil distinguir entre varias salas al mismo tiempo, los mensajes de cada sala se muestran en un color distinto en la terminal.

4. Conclusiones

Para concluir, puedo decir que este proyecto no fue nada fácil, ni tampoco imposible. Aprendí a usar y entender el nuevo lenguaje mientras lo iba necesitando, y pude ver los retos que implica construir un sistema de comunicación cliente-servidor, rigiéndome por un protocolo que debía seguir al pie de la letra, el cual tiene reglas explícitas sobre qué mensajes son válidos y qué responder a cada situación. Esto fue fundamental para que tanto los clientes como los servidores de otros compañeros pudieran ejecutarse entre sí sin importar el lenguaje en el que estuvieran hechos.

La programación concurrente fue uno de los mayores aprendizajes de este proyecto, donde hay que atender a cada cliente en su propio hilo, resolviendo el problema de servir a múltiples clientes de manera simultánea. La prueba con 100 usuarios conectados fue clave desde el inicio para saber en qué fallaba mi código, fallas que no se mostraban con pocos clientes; desde ahí pude apreciar el porqué de los tests. Y por último, decir que el MVC ayudó mucho a este tipo de proyecto, ya que se pudo tener mejor organizado y más limpio.

5. Bibliografía

- Documentación de Kotlin: <https://kotlinlang.org/docs/home.html>
- kotlinx.serialization: <https://github.com/Kotlin/kotlinx.serialization>
- Sockets en Java: <https://docs.oracle.com/javase/tutorial/networking/sockets/>
- Thread en Java: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html>

- JUnit 5: <https://junit.org/junit5/docs/current/user-guide/>
- Video: <https://www.youtube.com/watch?v=KlA84hlAmqg>
- Video: <https://www.youtube.com/watch?v=3wJTI9LM0sk>